

Transformer Anatomy

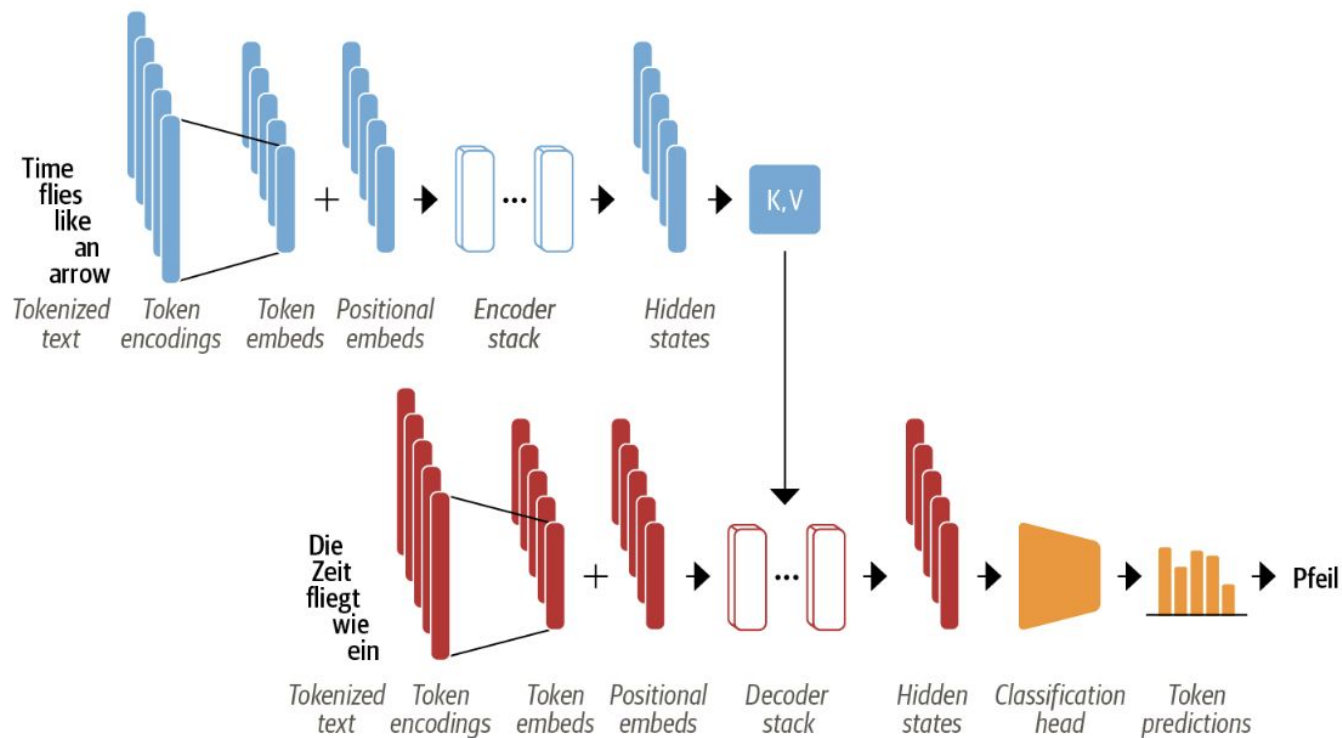
UCU NLP Course

Yurii Laba

Agenda?

The Transformer

Meet The Transformer!



Encoder-only

1. Input tokens → processed through stacked self-attention + feedforward layers in the encoder.
2. The model outputs contextualized embeddings for each token.
3. No autoregressive decoding - just a contextual representation.
4. Examples: BERT, RoBERTa, DistilBERT, ALBERT

Encoder-only

1. Input tokens → processed through stacked self-attention + feedforward layers in the encoder.
2. The model outputs contextualized embeddings for each token.
3. No autoregressive decoding - just a contextual representation.
4. Examples: BERT, RoBERTa, DistilBERT, ALBERT

Decoder-only

1. Uses only the decoder stack of the original Transformer.
2. Self-attention is causal (masked) so the model can't peek at future tokens.
3. Trained autoregressively: predict the next token given previous ones.
4. Examples: GPT family (GPT-2, GPT-3, GPT-4, GPT-4o, ChatGPT), LLaMA, Mistral, Falcon, etc.

Encoder-only

1. Input tokens → processed through stacked self-attention + feedforward layers in the encoder.
2. The model outputs contextualized embeddings for each token.
3. No autoregressive decoding - just a contextual representation.
4. Examples: BERT, RoBERTa, DistilBERT, ALBERT

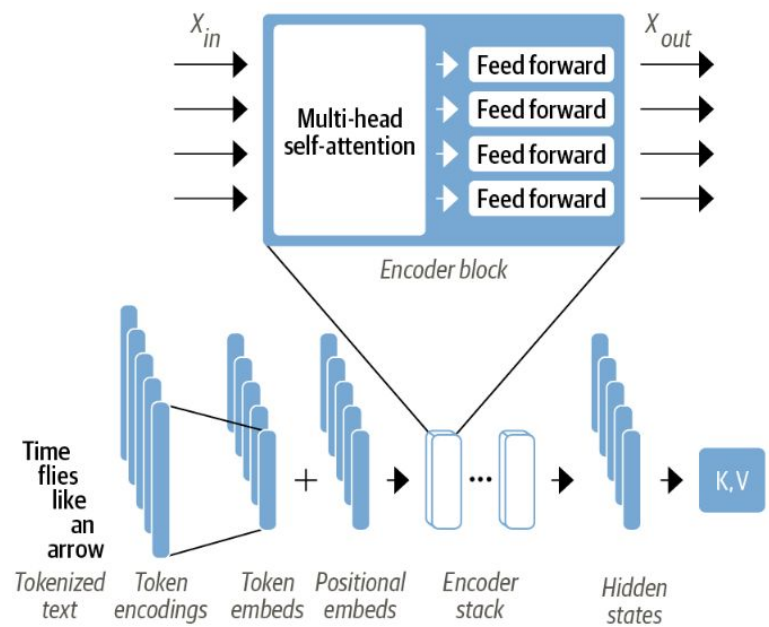
Decoder-only

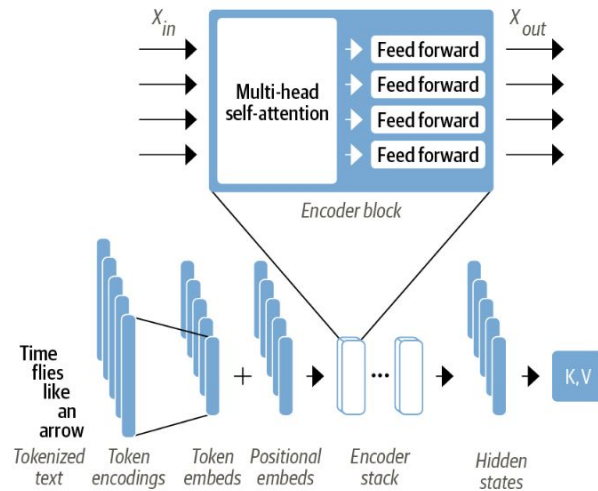
1. Uses only the decoder stack of the original Transformer.
2. Self-attention is causal (masked) so the model can't peek at future tokens.
3. Trained autoregressively: predict the next token given previous ones.
4. Examples: GPT family (GPT-2, GPT-3, GPT-4, GPT-4o, ChatGPT), LLaMA, Mistral, Falcon, etc.

Encoder-decoder

1. Encoder: Processes the full input sequence into contextual embeddings.
2. Decoder: Generates output tokens autoregressively, attending both to previously generated tokens and the encoder's representations.
3. This allows mapping one sequence to another.
4. Examples: T5 (Text-to-Text Transfer Transformer), BART, mBART, MarianMT (translation).

Transformer Encoder





1. The Transformer's encoder consists of many encoder layers stacked next to each other.
2. Each encoder layer receives a sequence of embeddings and feeds them through the following sub-layers:
 - a. A multi-head self-attention layer.
 - b. A feed-forward layer.
3. The main role of the encoder stack is to **"update" the input embeddings** to produce representations that encode some **contextual information** in the sequence.
 - a. For example the word "apple" will be updated to be more "company-like" and less "fruit-like" if the words "keynote" or "phone" are close to the word.

What is the self-attention?

Self-attention

Attention is a mechanism that allows neural networks to assign a different amount of weight or “attention” to each element in a sequence.

Self-attention

Attention is a mechanism that allows neural networks to assign a different amount of weight or “attention” to each element in a sequence.

The main idea behind self-attention is that instead of using a fixed embedding for each token, we can use the whole sequence to compute a weighted average of each embedding.

How to implement the self-attention?

There are several ways to implement a self-attention layer, but the most common one is **scaled dot-product** attention from the **Attention is All You Need** paper where the Transformer was introduced.

1. Each token embedding is projected into three vectors called **query, key, and value**.

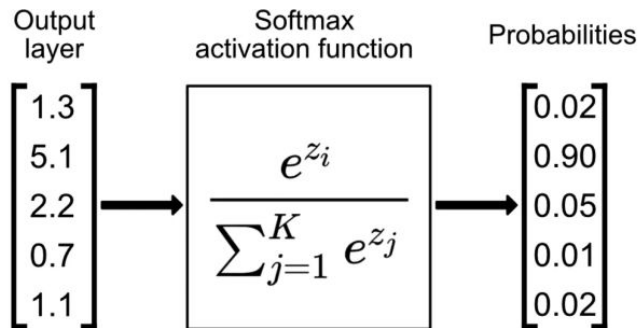
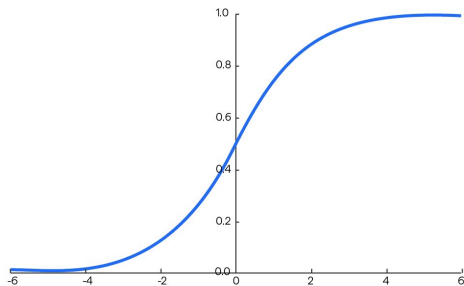
There are several ways to implement a self-attention layer, but the most common one is **scaled dot-product** attention from the **Attention is All You Need** paper where the Transformer was introduced.

1. Each token embedding is projected into three vectors called query, key, and value.
2. Determine how much the query and key vectors relate to each other using a **similarity function**. The outputs from this step are called the attention scores and for a sequence.

There are several ways to implement a self-attention layer, but the most common one is **scaled dot-product** attention from the **Attention is All You Need** paper where the Transformer was introduced.

1. Each token embedding is projected into three vectors called query, key, and value.
2. Determine how much the query and key vectors relate to each other using a similarity function. The outputs from this step are called the attention scores and for a sequence.
3. Dot-products can in general produce arbitrarily large numbers which can destabilize the training process. To handle this, the attention scores are first multiplied by a **scaling factor** to normalize their variance and then normalized with a **softmax** to ensure all the column values sum to one.

Softmax Function



There are several ways to implement a self-attention layer, but the most common one is **scaled dot-product** attention from the **Attention is All You Need** paper where the Transformer was introduced.

1. Each token embedding is projected into three vectors called query, key, and value.
2. Determine how much the query and key vectors relate to each other using a similarity function. The outputs from this step are called the attention scores and for a sequence.
3. Dot-products can in general produce arbitrarily large numbers which can destabilize the training process. To handle this, the attention scores are first multiplied by a scaling factor to normalize their variance and then normalized with a softmax to ensure all the column values sum to one.
4. Once the attention weights are computed, we multiply them by the **value vector**.

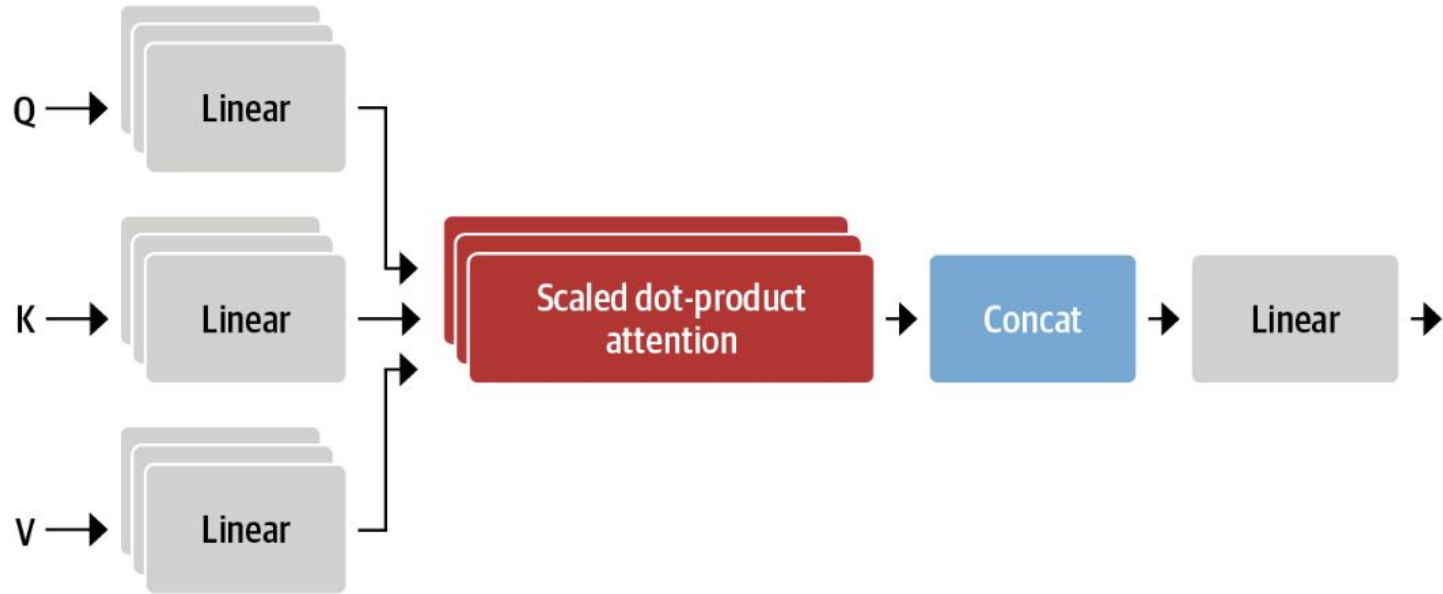
$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

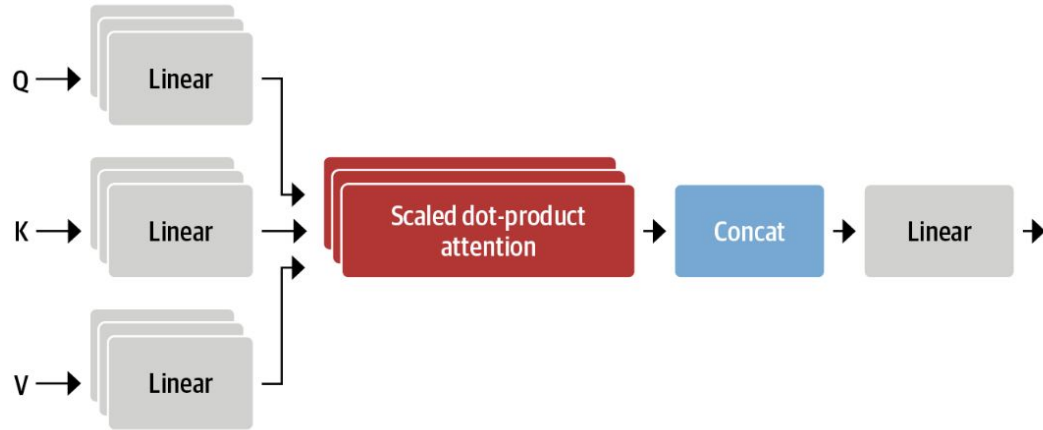
The self-attention calculation in matrix form

Is such initialization of Q , K and V good enough?

MultiHead Attention

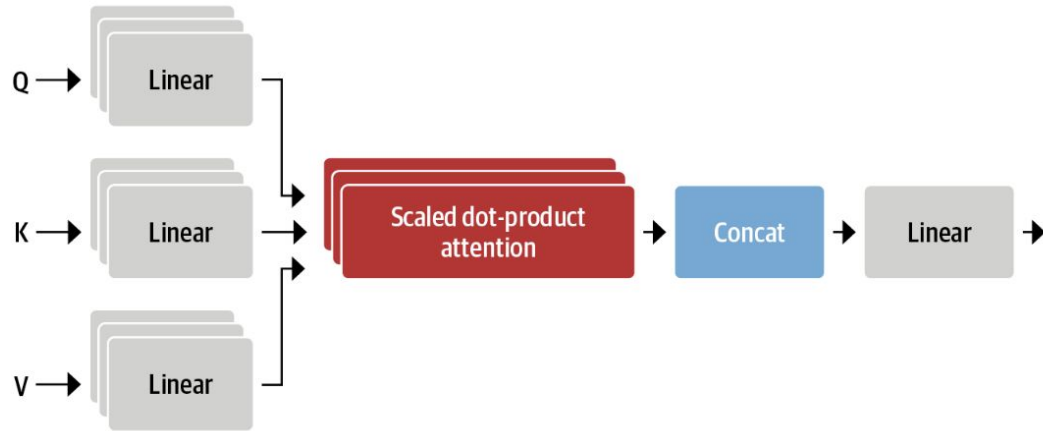


MultiHead Attention



But why do we need more than one attention heads?

MultiHead Attention

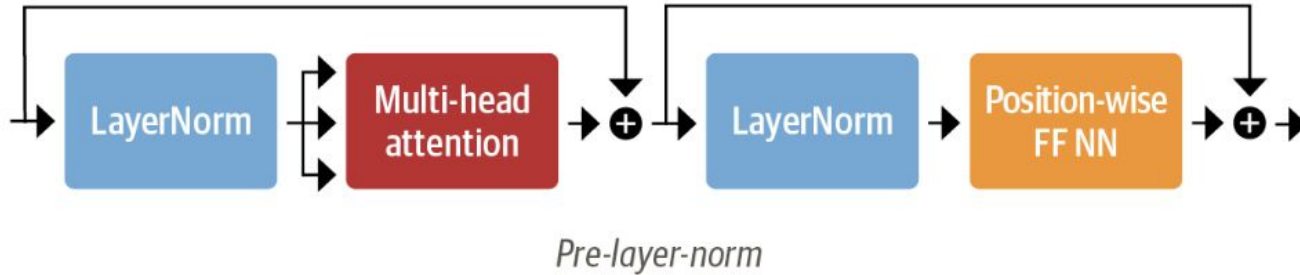
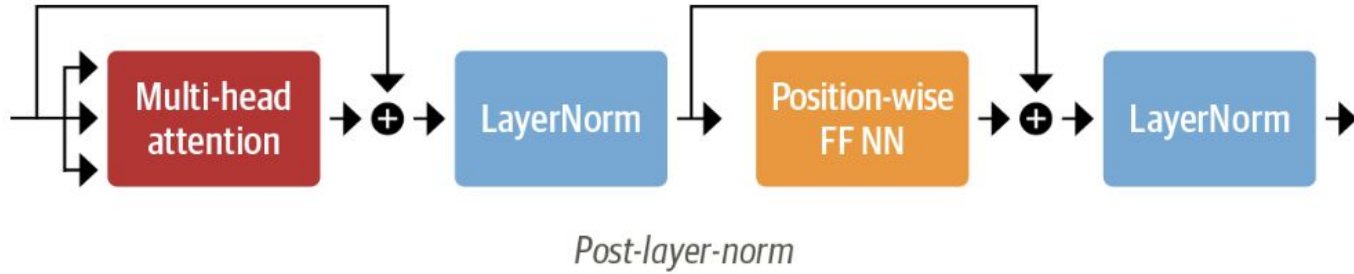


But why do we need more than one attention heads?

The reason is that softmax of one head tends to focus on mostly one aspect of similarity.

Feed Forward

Adding Layer Normalization



Positional Embeddings

```
class TransformerForSequenceClassification(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.encoder = TransformerEncoder(config)
        self.dropout = nn.Dropout(config.hidden_dropout_prob)
        self.classifier = nn.Linear(config.hidden_size, config.num_labels)

    def forward(self, x):
        x = self.encoder(x)[: , 0 , :] # select hidden state of [CLS] token
        x = self.dropout(x)
        x = self.classifier(x)
        return x
```